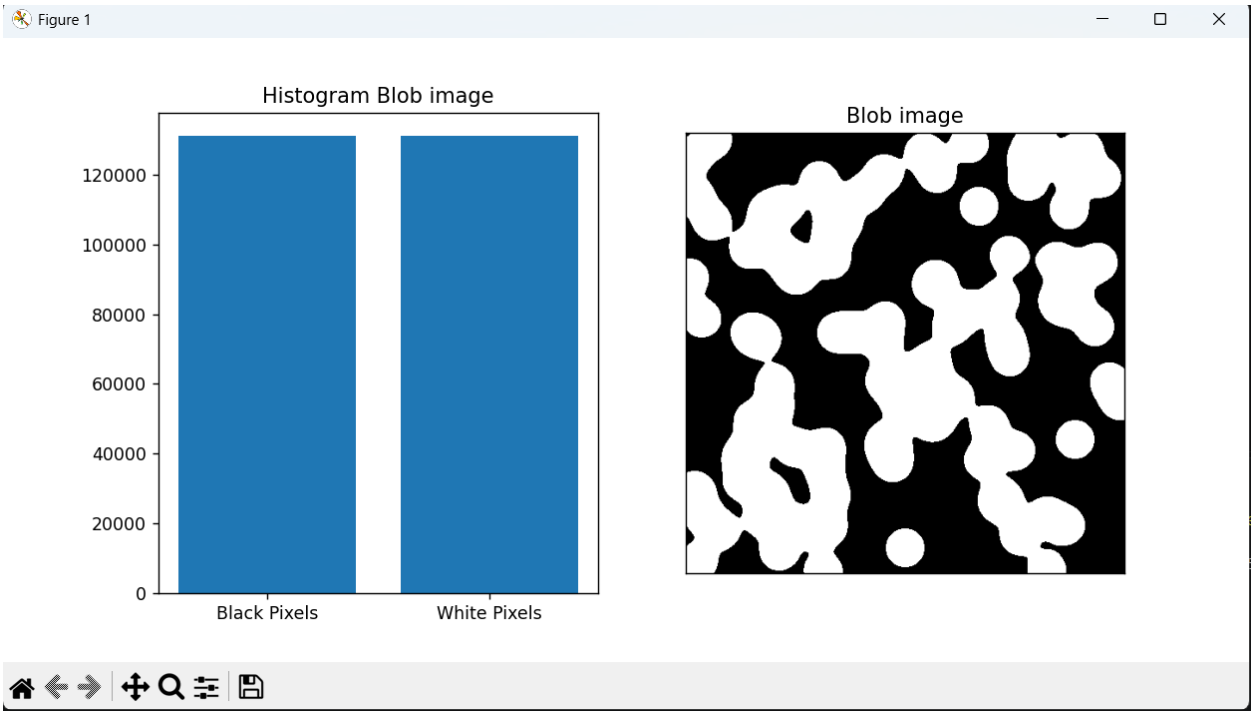
Assignment 2

By Brage Bestvold and Lasse Lindholm

Question 1: Display the histogram of a binary image 'Blob'.

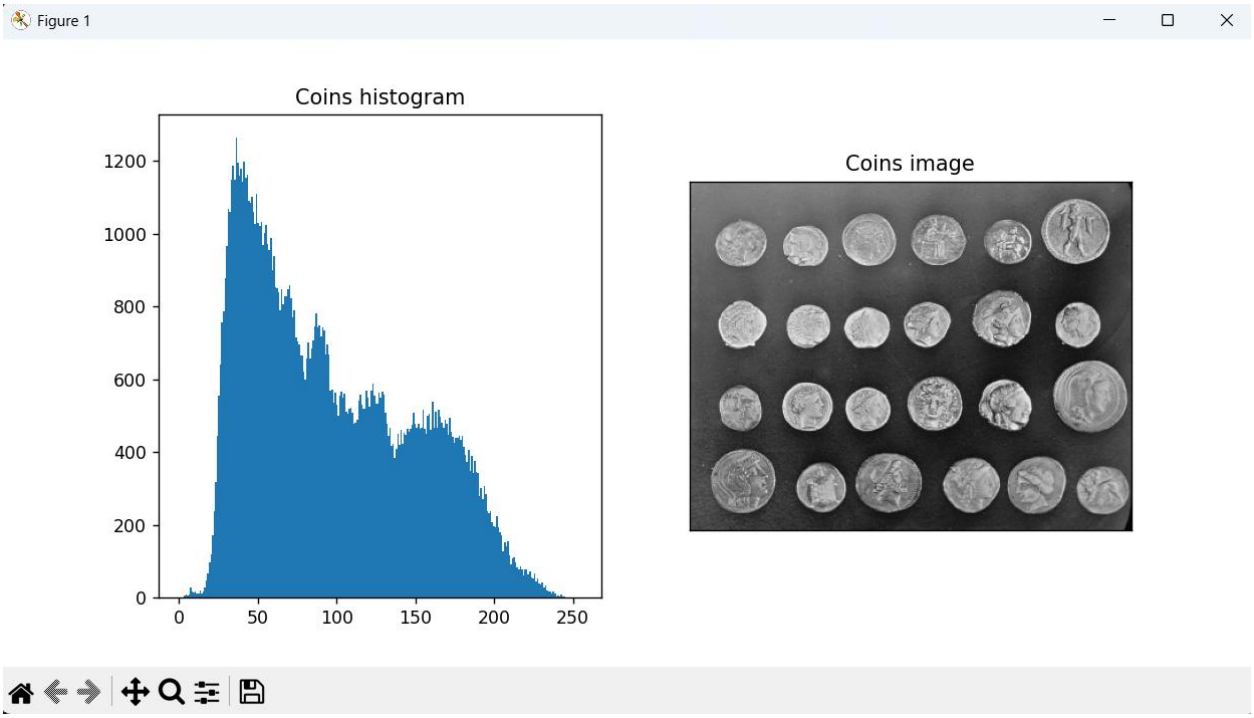
- Loads data.binary_blobs image.
- Converts it from True/False to the standard 0/255 black & white convention, so the histogram x-axis is easy to interpret.
- Manually counts black vs. white pixels using np.sum() (since True counts as 1 in a sum), just to cross-check the histogram visually.
- Plots the image next to its histogram, using bins=2 — because a true binary image only ever has two possible pixel values, so only two bars will appear.
- Displays both side by side for direct visual comparison.
- Calculate and print what PERCENTAGE of the image is white (horse) vs. black (background)




```
Question 1
50.0% of the image is white
50.0% of the image is black
********************************
```

Question 2: Display the histogram of gray image 'Coin'.

- Load data.coins().
- Confirm its dtype (uint8) and pixel value range (0–255) before plotting.
- Plot the image next to its histogram, using bins=256 — one bin for every possible gray level from 0 to 255, since (unlike the binary case) any of these 256 values could genuinely appear.
- Add a short block at the end to compute the mean and mode pixel values from the image.

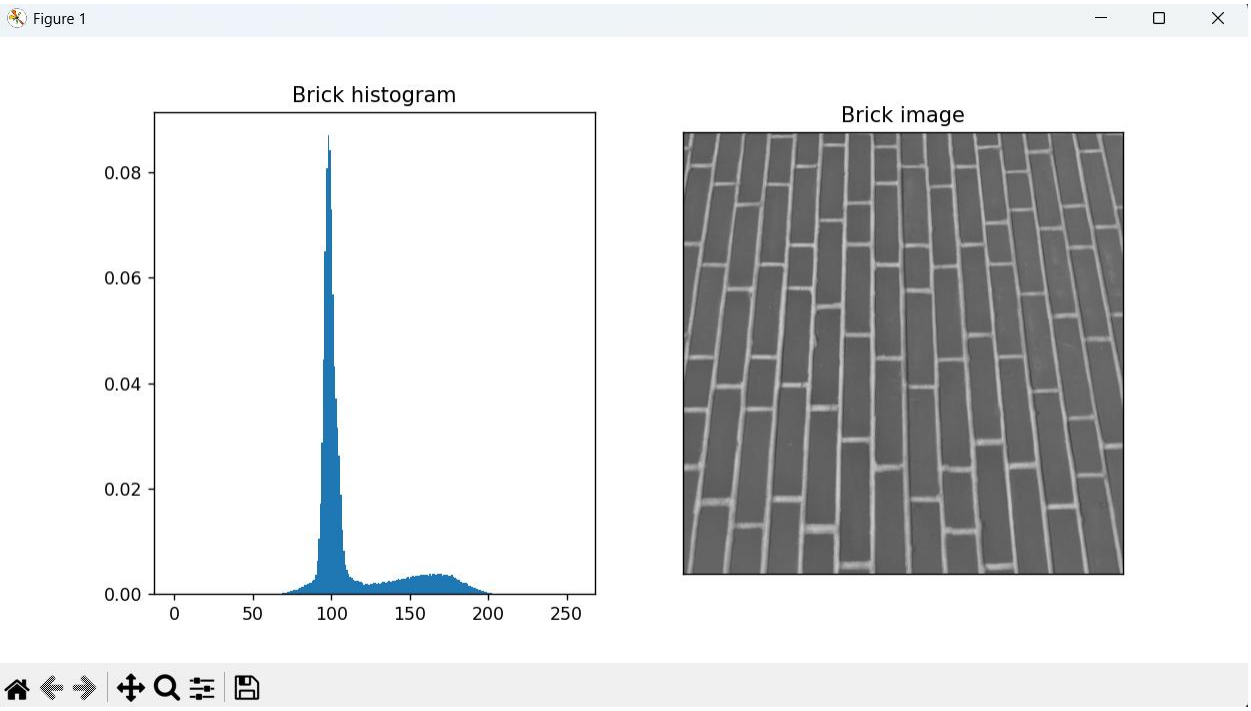



```
Question 2
Datatype: uint8
Min value: 1
Max value: 252
All pixels lie in the range from 0 to 255
Mean pixel value: 96.85551602035204
Mode: 36
******************************
```

Question 3: Display the normalized histogram of gray image 'Brick'

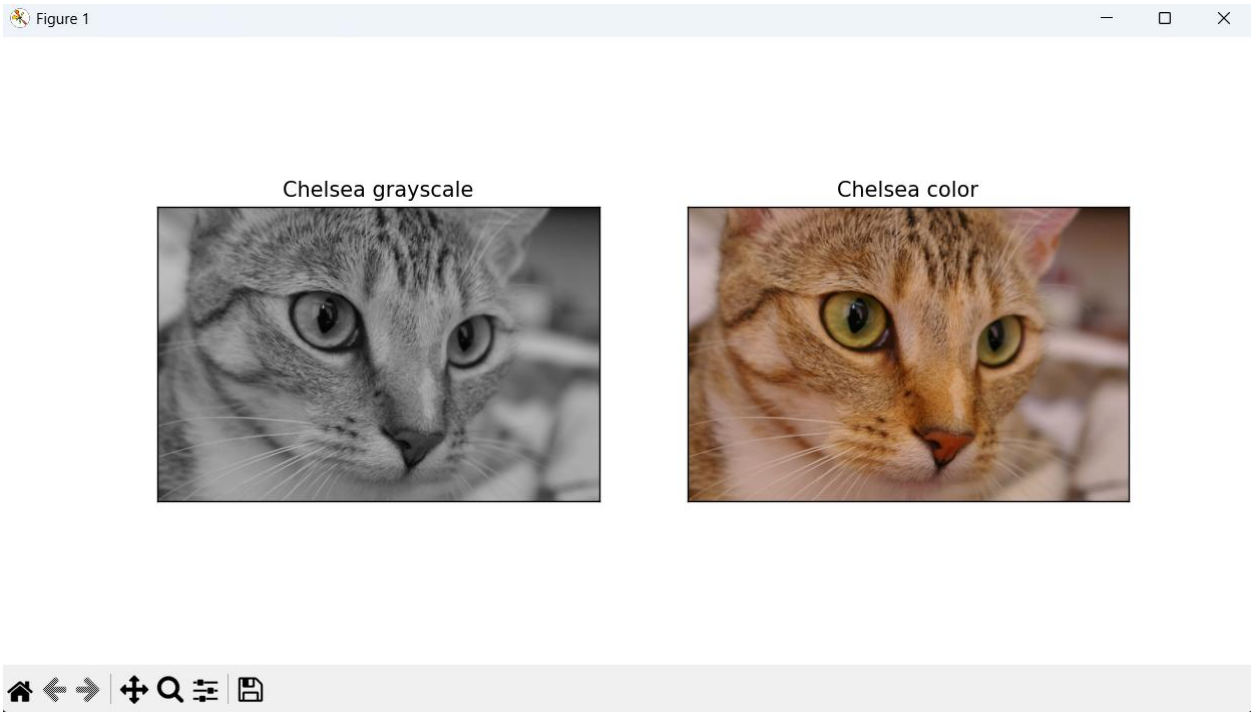
- Load data.brick() image.
- Plot it next to its histogram, but this time with density=True passed into plt.hist().

- Instead of showing raw pixel counts on the y-axis, it shows probability density — the histogram is rescaled so the total area under all bars sums to exactly 1



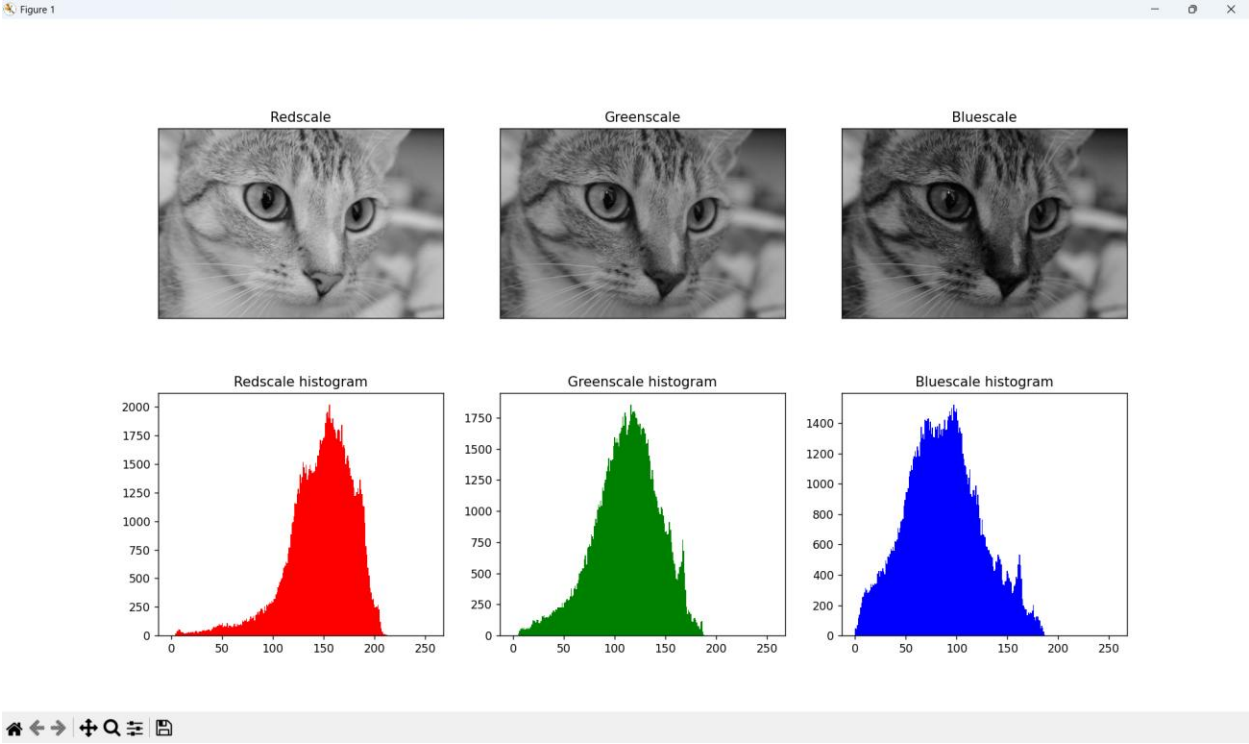
Question 4: Convert RGB image 'Chelsea' to gray scale image

- Loads the RGB chelsea image and splits it into its R, G, and B channel arrays.
- Applies the luminosity formula Y = w_R·R + w_G·G + w_B·B using the ITU-R BT.709 weights — the standard used for modern displays.
- Converts channels to float64 before the weighted sum to avoid uint8 overflow, then clips and converts back to uint8 for standard image display.
- Displays the original RGB image next to the resulting single-channel grayscale image.



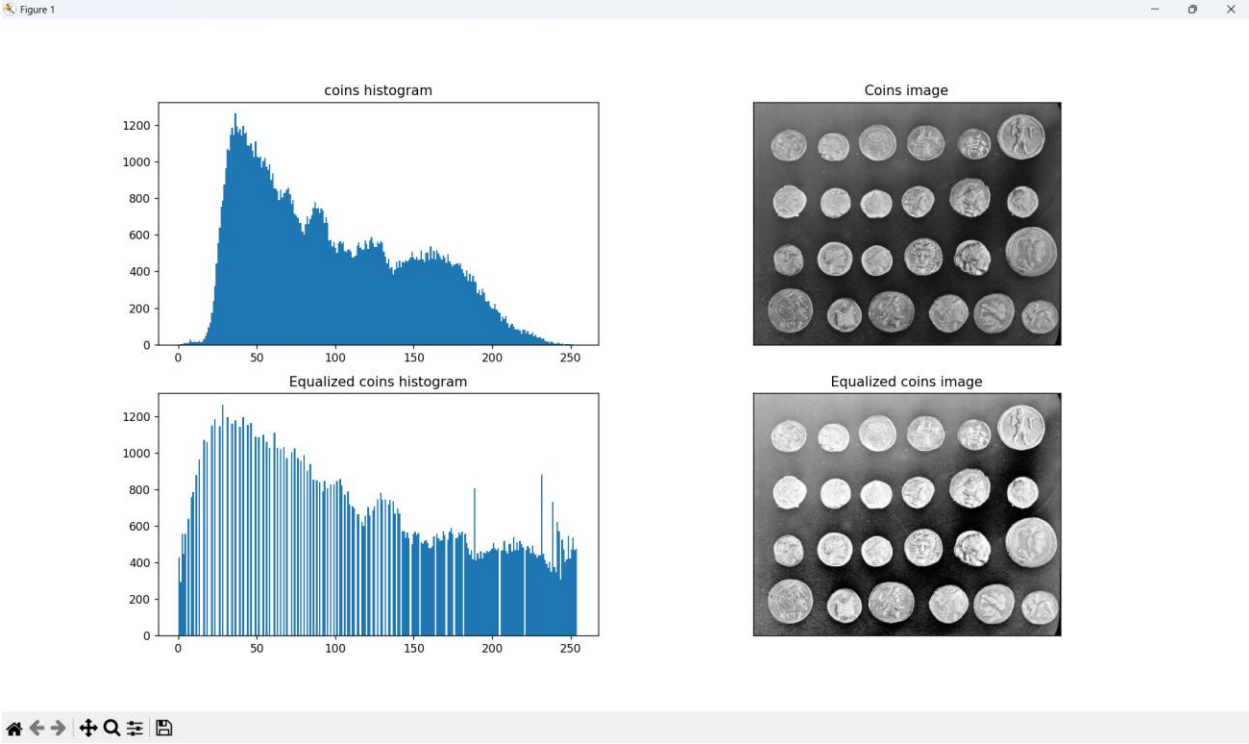
Question 5: Disply the histogram of RGB image 'chelsea', and the histogram of its individual channels.

- Loads the RGB chelsea image and splits it into R, G, B channels.
- Computes a luminance image using the weighted BT.709 formula, then displays it alongside the original RGB image and its histogram.
- Displays each of the R, G, B channels individually in grayscale (cmap="gray"), so you're viewing pure intensity per channel.
- Plots the histogram of each channel separately, using a matching bar color (red/green/blue) purely as a visual label — the underlying pixel values themselves are just numbers 0–255, regardless of bar color.



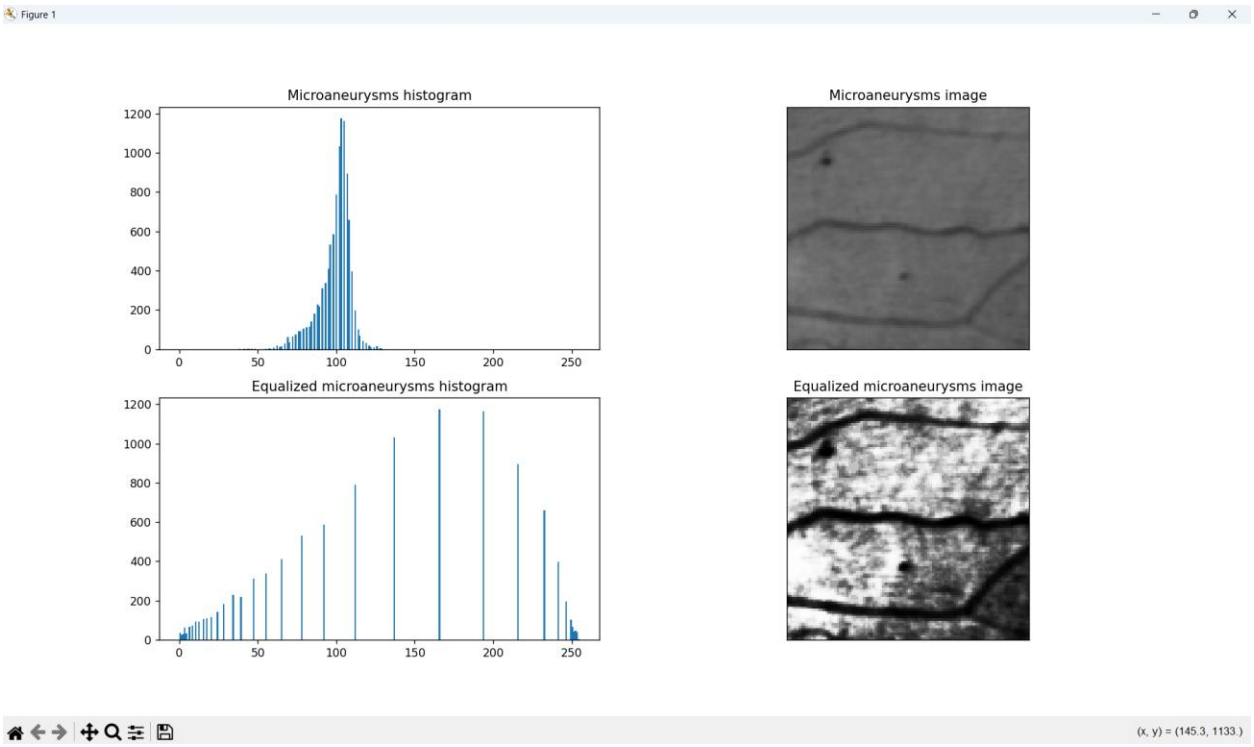
Question 6. Display the gray scale 'Coins' image and its histogram. Apply histogram equalization, display the modified image and its histogram.

- Load data.coins()
- Apply exposure.equalize_hist() — a function that redistributes pixel intensities so the histogram becomes as flat/uniform as possible, which boosts contrast and brings out hidden detail in dark regions.
- Convert the equalized result from equalize_hist()'s native float64 (0.0–1.0) output back into the standard uint8 (0–255) format.
- Display original image, original histogram, equalized image, equalized histogram — so you can directly compare before/after.



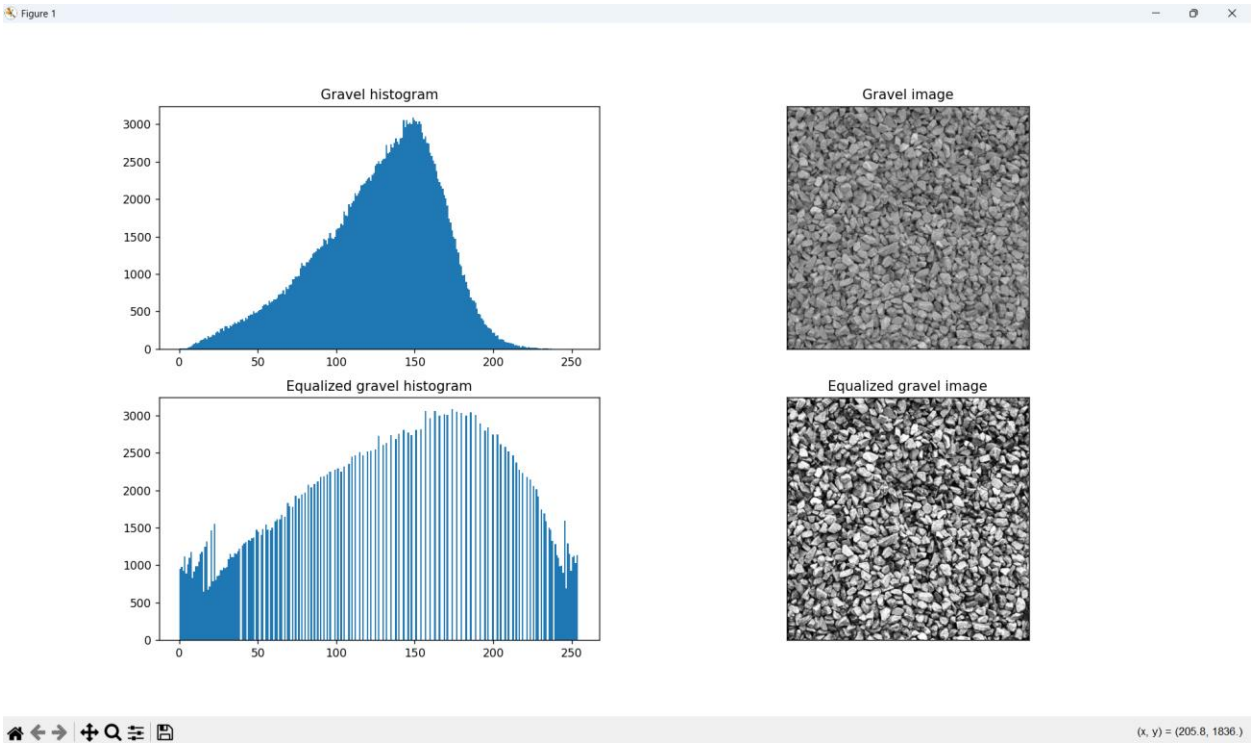
Question 7: Display the gray scale 'microaneurysms' image and its histogram. Apply histogram equalization, display the modified image and its histogram.

- Loads data.microaneurysms() which is a cropped retina image used in medical imaging.
- Since this function returns float64 values (0.0–1.0) by default, it's first converted to standard uint8 (0–255) so it behaves consistently with the other grayscale examples.
- Apply exposure.equalize_hist() to redistribute pixel intensities toward a flatter, more uniform histogram — boosting contrast.
- Convert the equalized result back to uint8 for standard display.
- Display original vs. equalized image and their histograms.



Question 8: Disply the gray scale 'Gravel' image and its histogram. Apply histogram equalization, display the modified image and its histogram.

- Load data.gravel()
- Apply exposure.equalize_hist() to redistribute pixel intensities toward a flatter, more uniform histogram, which can help normalize the uneven lighting across the page and sharpen the contrast between text and background.
- Convert the equalized result back to uint8 for standard display.
- Displays original image, original histogram, equalized image, equalized histogram — so you can directly compare before/after.


Figure 1
Gravel histogram
Gravel image
Equalized gravel histogram
Equalized gravel image
0
500
1000
1500
2000
2500
3000
0
50
100
150
200
250
(x, y) = (205.8, 1836.)